

TALLINN UNIVERSITY OF TECHNOLOGY

School of Information Technologies

Robin Rebane 244040IADB

Realms of Ash

Project

Supervisor: Andres Käver

Tallinn 2026

List of abbreviations and terms

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ERD	Entity Relationship Diagram
FK	Foreign Key
JSONB	JSON Binary (PostgreSQL binary storage format for JSON)
JWT	JSON Web Token
MVC	Model-View-Controller
PK	Primary Key
REST	Representational State Transfer
SignalR	ASP.NET Core library for real-time web functionality via Web-Sockets
SPA	Single-Page Application
UUID	Universally Unique Identifier

Table of contents

1	Introduction.....	5
1.1	Why Does Realms of Ash Matter?.....	5
2	Main Features	7
3	Main User Flows	9
3.1	Player / Kingdom Owner Flow	9
3.2	Administrator Flow	9
3.3	General Discovery and Match-Setup Flow	10
4	Database Schema	11
4.1	Identity	11
4.2	Reference Data	11
4.3	Game and Lobby	12
4.4	Map	12
4.5	Buildings and Armies.....	12
4.6	Resources	12
4.7	Combat.....	12
4.8	Audit	13
5	Summary	14

1 Introduction

Realms of Ash is a turn-based, multiplayer kingdom-building strategy game delivered as a web application. Up to four players share a procedurally generated hex-tile map, each commanding an independent medieval fantasy kingdom. Over the course of a session—designed to conclude in approximately one hour—players expand their territory, develop a five-chain resource economy, construct a tiered network of buildings, recruit and deploy armies, and wage tactical battles to eliminate rival kingdoms. Private lobbies with a six-character invite code keep sessions friendly and accessible: no matchmaking queue, no ranked ladder.

The application is built on a clean-architecture .NET 10 backend exposing a versioned REST API and a SignalR real-time hub, paired with a React 19 / TypeScript single-page frontend rendered on an HTML5 canvas. A parallel server-rendered Razor client (Public MVC area) provides full feature parity for environments where JavaScript is unavailable or restricted. All game state is persisted in PostgreSQL; live events—turn advances, battle outcomes, slot-machine spins—are pushed to connected clients via SignalR without any polling.

1.1 Why Does Realms of Ash Matter?

Browser-based strategy games that combine meaningful economic depth with real-time social coordination are rare at the hobby and student-project scale. Most web games either reduce to simple clicker mechanics or require dedicated desktop clients. Realms of Ash occupies the gap: a full-featured turn-based strategy experience—hex map, asymmetric factions, five resource chains, multi-round tactical combat—delivered entirely through a standard web browser with no installation.

From a technical standpoint the project addresses several non-trivial engineering challenges simultaneously. Concurrent multi-player writes to shared game state require per-game

mutex locking (via a named SemaphoreSlim manager) and optimistic concurrency via PostgreSQL's native xmin row-version column. A background service polls for expired turn deadlines every fifteen seconds, auto-advancing stale turns and cleaning up abandoned lobbies without operator intervention. The combat system resolves battles deterministically round-by-round—initiative ordering, damage ranges, chip damage, situational bonuses, faction modifiers—entirely in the domain layer with no external dependencies. Map generation produces an anti-clustered hex terrain layout that guarantees each player receives a castle tile and seven contiguous starting tiles regardless of player count or map dimensions.

Building the project also required maintaining two fully functional client surfaces simultaneously: the canvas-based React SPA and the server-rendered Razor MVC client. Both clients share the same API, the same SignalR hub, and the same authentication pipeline, demonstrating that a clean separation between application logic and presentation layer enables multiple UI paradigms from a single backend.

2 Main Features

This chapter describes the core features of Realms of Ash. Each feature is implemented as a distinct system within the application and contributes to the overall game experience.

- **Private lobby system.** Players create a lobby and share a six-character invite code. Each player selects one of four asymmetric factions before the host starts the game. Faction selection is broadcast in real-time via SignalR so all lobby members see changes immediately.
- **Procedural hex map generation.** A hex grid of configurable dimensions is generated with anti-clustering terrain distribution across five terrain types (Plains, Forest, Mountain, Desert, Magic Grove). Each player receives a castle tile and seven surrounding starting tiles; the rest of the map is unclaimed and contested.
- **Four-phase round cycle.** Each round progresses through Action → Battle → Income → Round End phases. Phase transitions are enforced server-side and broadcast to all clients.
- **Action phase economy.** Players spend action points (base four per turn, faction-modified) to place or upgrade buildings from six tiered chains, train one of six army types at military buildings, or gamble at the slot machine for a random action-point adjustment.
- **Five-resource economy.** Gold, Food, Wood, Stone, and Mana are produced each Income phase by owned buildings, scaled by terrain-type yield multipliers and faction production modifiers. Army upkeep and healing are also applied during Income. Resources are tracked to two decimal places per kingdom.
- **Six building chains.** Each chain has three upgrade tiers (e.g., Farm → Windmill → Granary for food). Placing a tier-one building automatically claims adjacent unclaimed tiles. A self-referencing unlock tree in reference data governs upgrade eligibility.
- **Tactical turn-based combat.** A player in the Action phase may declare an attack on an adjacent tile. Both attacker and defender independently select armies (up to

three; up to five if the target is a castle), set their lineup order, and submit. When both lineups are ready the server resolves combat deterministically round-by-round: initiative ordering, damage ranges, chip damage, situational bonuses, and faction modifiers are all applied in domain logic. The outcome tile transfers to the winner; the loser's fallen armies are destroyed.

- **Elimination win condition.** Capturing a kingdom's castle tile eliminates that player. The last remaining kingdom wins. Final standings and elimination order are recorded and displayed after the game ends.
- **Turn timeout background service.** An optional per-game turn time limit triggers automatic turn advancement when a player's deadline expires. A hosted `BackgroundService` polls every fifteen seconds. Kingdoms that miss turns consecutively can be flagged for stale-lobby cleanup.
- **Slot machine.** Costs 30 Gold to spin. Weighted outcomes adjust action points by -2 , -1 , 0 , $+1$, or $+2$ for the current turn. Weights are stored as JSONB on the `Game` entity and are configurable by administrators through the admin panel.
- **Admin panel.** A cookie-authenticated MVC area at `/root/` gives administrators full CRUD over reference data (factions, terrain types, building types, army types), user management (role assignment, account locking), and the ability to force-close stuck lobbies.
- **Bilingual localization.** All user-facing strings in both the API and the MVC client are localized to English and Estonian via `.resx` resource files. The active culture is set via a cookie; the `Accept-Language` header is used as fallback.

3 Main User Flows

This chapter describes the main user flows in Realms of Ash, covering the experience of regular players, administrators, and unauthenticated visitors.

3.1 Player / Kingdom Owner Flow

- Register or log in and land on the lobby home page.
- Create a lobby, receive a six-character invite code, and share it with up to three other players.
- Wait in the lobby; select a faction (Iron Throne, Mage Council, Merchant Republic, or Forest Elves); see other players' faction choices update live.
- The host clicks Start (requires two or more players, all with factions selected): the game is initialized with the map generated, kingdoms created, and turn order assigned.
- **Action phase (per turn):** view the hex map; build or upgrade buildings; train armies; spin the slot machine; declare attacks on adjacent rival tiles; end turn.
- **Battle phase:** if an attack was declared, select armies and set lineup order; wait for the opponent to do the same; watch the battle resolve round-by-round with the outcome broadcast to all players.
- **Income phase:** resources are credited automatically; upkeep is deducted; armies heal; the win condition is checked.
- Continue until one kingdom captures all rivals' castles; the game over screen shows final standings and elimination order.
- Optionally abandon an active game; if the last player abandons, the game is closed.

3.2 Administrator Flow

- Navigate to /root/ and log in with admin credentials (separate cookie-based authentication scheme).

- Manage users: view the list, assign or revoke the Admin role, lock or unlock accounts.
- Manage active games: view the status of all in-progress lobbies and force-close stuck sessions.
- Edit reference data: update faction modifiers, terrain yield multipliers, building type stats, army type stat profiles, and slot machine weights on any active game.

3.3 General Discovery and Match-Setup Flow

- Unauthenticated visitors land on the public home page and can view a description of the game.
- After registration, a player immediately sees their active game (if any) and can rejoin mid-session.
- Real-time SignalR events keep all connected clients in sync throughout the lobby and game lifecycle without page reloads or manual polling.

4 Database Schema

The database schema underpinning Realms of Ash is implemented in PostgreSQL. All domain entities inherit a `BaseEntity` providing a UUID primary key (`Id`), `CreatedAt`, and `UpdatedAt` timestamps. Identity tables (`AspNetUsers`, `AspNetRoles`, `AppRefreshToken`, `DataProtectionKeys`) are managed by ASP.NET Core Identity and appear at the domain boundary where domain entities reference them. Full ERD diagrams (all columns and PK/FK-only compact view) are available as separate SVG files alongside this document.

4.1 Identity

`AppUser` extends the ASP.NET Core Identity user with application-specific fields. `AppRefreshToken` stores JWT refresh tokens linked to users, each with an expiry timestamp and a revocation flag.

4.2 Reference Data

Four reference tables define the static game configuration that can be edited by administrators via the admin panel.

- **FactionType** — Defines the four asymmetric factions. Stores numeric modifiers for attack, HP, initiative, chip damage, resource production, building cost, training cost, action points, and heal rate, as well as a starting resource bonus type and amount.
- **TerrainType** — Defines the five terrain types (Plains, Forest, Mountain, Desert, Magic Grove) with a resource bonus type and a yield multiplier.
- **BuildingType** — Defines building tiers and resource chains, storing production stats, build costs in Gold/Wood/Stone, a military flag, and a self-referencing `UnlockedByBuildingTypeId` foreign key that forms the upgrade eligibility tree.
- **ArmyType** — Defines the six unit types with full combat statistics (`Attack`, `MaxHp`, `Initiative`, `DamageMin`, `DamageMax`, `ChipDamageMin`, `ChipDamageMax`, `Situa-`

tionalBonus), training costs, upkeep costs, and a required building type.

4.3 Game and Lobby

Game is the central aggregate. It records the lobby code, current status (*Lobby*, *InProgress*, or *Completed*), the active round phase (*Action*, *Battle*, *Income*, or *RoundEnd*), map dimensions, slot machine weight configuration (stored as JSONB), turn deadline, and the winning kingdom. Optimistic concurrency is achieved via the PostgreSQL native `xmin` row-version column.

Kingdom represents a player's in-game presence. It links an `AppUser` and a `FactionType` to a `Game`, and records turn order, status (*Active* or *Defeated*), remaining action points, and consecutive missed turns.

4.4 Map

Tile models each cell of the hex grid. It stores axial coordinates (Q, R), terrain type, an optional owning kingdom, and a flag indicating whether a castle occupies the tile.

4.5 Buildings and Armies

Building records a placed building instance on a tile; each tile may host at most one building. Army records a unit stationed at a building, tracking both current and maximum HP.

4.6 Resources

KingdomResource tracks each of the five resource types (Gold, Food, Wood, Stone, and Mana) per kingdom as individual rows, with amounts stored to two decimal places.

4.7 Combat

DeclaredAttack captures an in-progress attack declaration, recording attacker and defender kingdoms and tiles along with a readiness flag for each side. Once both sides submit lineups, the server resolves the battle and writes a `Battle` record storing the

outcome and the captured tile (nullable). Each battle is decomposed into `BattleRound` records that store per-round statistics as a JSON payload.

4.8 Audit

`TurnLog` records game events—building placed, army trained, attack declared, etc. —as structured JSON payloads, providing a full audit trail of each kingdom’s actions per round.

5 Summary

Realms of Ash is a web-based turn-based multiplayer strategy game built on .NET 10 and React 19 that demonstrates clean-architecture design across a REST API, a SignalR real-time hub, and two parallel client surfaces. Key engineering challenges addressed include concurrent game-state management with optimistic concurrency, procedural hex map generation with anti-clustered terrain distribution, deterministic round-by-round combat resolution, bilingual localization, and a background service for automated turn advancement. The PostgreSQL schema covers eleven domain entity groups spanning identity, reference data, game state, map, economy, combat, and audit.